

Clase del Módulo 2

Sitio: [Campus - Sadosky Capacitaciones](#)

Curso: Introducción al mundo de la programación paralela para
HPC_V1

Libro: Clase del Módulo 2

Imprimido por: Pablo Carrai

Día: jueves, 4 de junio de 2026, 10:07

Tabla de contenidos

- 1. Introducción
- 2. Directivas básicas y estructura de un programa paralelo con OpenMP
 - 2.1. Compilación y ejecución de programas con OpenMP
 - 2.2 Control de threads y regiones paralelas
 - 2.3 Directiva master y single
- 3. Sincronización y comunicación
- 4. Directivas Avanzadas
- 5. Distribución de trabajo (work-sharing)
 - Sections: paralelismo de tareas independientes
 - Cierre

1. Introducción

En la era de la computación multicore y los sistemas de HPC, la capacidad de aprovechar el paralelismo disponible en el hardware ya no es una opción, sino una necesidad. Los procesadores actuales integran múltiples núcleos de ejecución y la programación secuencial tradicional resulta insuficiente para explotar todo su potencial.

La programación con OpenMP (*Open Multi-Processing*) surge como una de las herramientas más accesibles, potentes y estandarizadas para desarrollar aplicaciones paralelas en arquitecturas de memoria compartida.

OpenMP permite que los programadores amplíen sus programas secuenciales en C, C++ o Fortran mediante un conjunto de directivas sencillas y portables, que instruyen al compilador para distribuir el trabajo entre múltiples hilos de ejecución (*threads*).

OpenMP representa un puente natural entre la programación secuencial y el paralelismo de alto rendimiento. A diferencia de otras librerías más complejas (como MPI o CUDA), su curva de aprendizaje es gradual: permite transformar programas existentes añadiendo pocas líneas de código.



Veamos un fragmento del siguiente video (minuto 1:42 a 5:00) que explica de manera sintética el funcionamiento de esta herramienta.

Sistemas Operativos UdeA



Mirar en

Sistemas operativos UdeA (24 de septiembre de 2020). Introducción a OpenMP [Video]. YouTube.



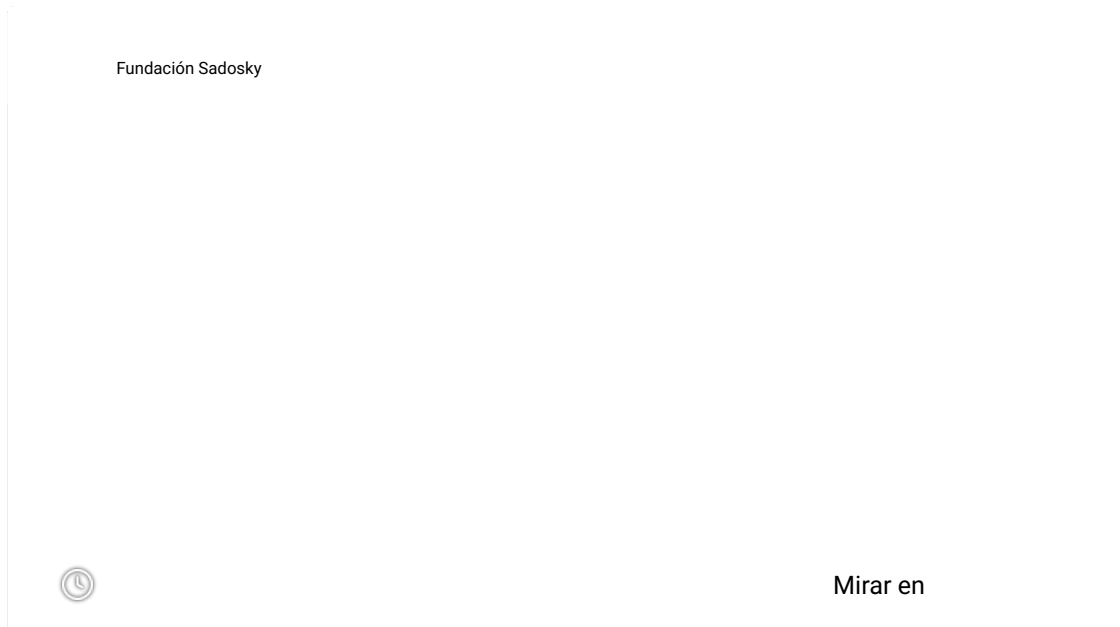
2. Directivas básicas y estructura de un programa paralelo con OpenMP

Como vimos en la introducción al módulo, este curso pretende ofrecer herramientas para que demos nuestros primeros

pasos en la programación paralela. Para eso, la profesora Verónica Gil Costa, autora de este curso, nos acompañará con una serie de videos tutoriales.



En este primer video, nos propone escribir el primer código en OpenMP, compilarlo y ejecutarlo. ¡Te invitamos a poner en práctica el paso a paso de este primer Hola Mundo a partir de ahora!



Se debe incluir en el código:

```
#include <omp.h>
```

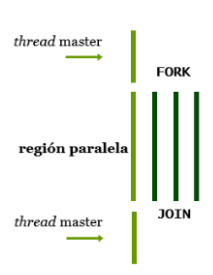
Las directivas de OpenMP se especifican de la siguiente manera, para C/C++:

```
#pragma omp <directiva>
```



`#pragma` es palabra reservada de OpenMP para especificar que se quiere utilizar una directiva.

El modelo de programación paralela que aplica OpenMP es *Fork - Join*.



Modelo *fork - join*:

- Un *thread* maestro lanza (*fork*) los *threads*.
- Cada *thread* ejecuta el mismo código y utilizan memoria compartida para comunicarse. Los *threads* pueden tener datos locales; utilizan instrucciones especiales y memoria compartida para sincronización.
- Finalmente, todos los *threads* se juntan (*join*) en el maestro.

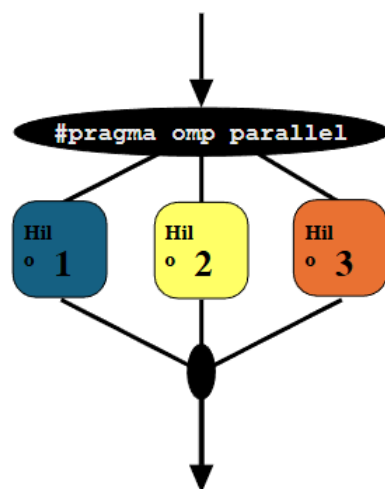
Todos los *threads* ejecutan la misma copia del código con diferentes datos, y a cada uno se le asigna un identificador (tid) para diferenciar las tareas ejecutadas:

```
if (tid == 0) then ... else ...
```

Constructor de paralelización

La siguiente función define una región paralela sobre un bloque de código estructurado. Los *threads* se crean como *parallel* y se bloquean al final de la región. Los datos se comparten entre *threads* a menos que se especifique otra cosa:

```
#pragma omp parallel{
    structured block
}
```



2.1. Compilación y ejecución de programas con OpenMP

Para compilar usamos:

```
g++ código.cc -o ejecutable -fopenmp
```

Variables de entorno:



OMP_NUM_THREADS indica el número de threads a usar. Dependiendo de la versión de Linux se utiliza:

- export OMP_NUM_THREADS=4
- setenv OMP_NUM_THREADS 4
- set OMP_NUM_THREADS=4

Veamos el ejemplo 1:

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        printf("Hola desde el thread %d\n", tid);
    }
    return 0;
}
```



La directiva `#pragma omp parallel` indica al compilador que las instrucciones dentro del bloque deben ejecutarse en paralelo por múltiples *threads*. Cada *thread* obtiene su identificador único mediante la función:
`omp_get_thread_num()`

Para compilar:

```
g++ hola_omp.c -o hola_omp -fopenmp
```

Y para ejecutar:

```
./hola_omp
```

La salida mostrará:

- Hola desde el *thread* 1
- Hola desde el *thread* 0
- Hola desde el *thread* 2
- Hola desde el *thread* 3

Para controlar el número de *threads* utilizamos:

```
export OMP_NUM_THREADS=6
```

Y para visualizar el número de *threads*:

```
echo $OMP_NUM_THREADS
```

2.2 Control de threads y regiones paralelas

En OpenMP, el modelo de ejecución se basa en la creación y manejo dinámico de hilos (*threads*) que ejecutan regiones del programa de manera simultánea.

Como se mencionó anteriormente, una región paralela es un bloque de código que será ejecutado por múltiples *threads*

en paralelo. Se define con la directiva:

```
#pragma omp parallel
{
    // Código a ejecutar por todos los threads
}
```

Al entrar en la región paralela:

- OpenMP crea un equipo (*team*) de *threads*.
- Cada *thread* ejecuta una copia del bloque de código.
- Al finalizar, todos los *threads* se sincronizan y el programa retorna a ejecución secuencial.
- El orden no está garantizado: todos los *threads* se ejecutan en paralelo.

El *thread* principal (llamado *master*) forma parte del equipo, y su identificador (`omp_get_thread_num()`) es 0.

Por defecto, OpenMP crea tantos threads como núcleos disponibles en el sistema. Sin embargo, el programador puede modificar este número de tres formas complementarias:

a) Con variable de entorno como se explicó anteriormente. Antes de ejecutar el programa:

```
export OMP_NUM_THREADS=4
```

b) Con función de biblioteca. Dentro del programa, antes de la región paralela usar:

```
omp_set_num_threads(4);
```

Ejemplo 2:

```
#include
#include
int main() {
    omp_set_num_threads(4);
    #pragma omp parallel
    {
        printf("Soy el thread %d de %d\n", omp_get_thread_num(),
            omp_get_num_threads());
    }
    return 0;
}
```

c) Con parámetro en la directiva:

```
#pragma omp parallel num_threads(4)
{
    printf("Soy el thread %d de %d\n", omp_get_thread_num(), omp_get_num_threads());
}
```

Si OpenMP no puede crear el número solicitado (por limitaciones del sistema), usará un número menor.

La cantidad de *threads* puede variar entre diferentes regiones paralelas del mismo programa. Dentro de una región paralela se pueden obtener datos sobre el entorno de ejecución mediante funciones de la librería OpenMP.

Función	Descripción
<code>int omp_get_thread_num()</code>	Devuelve el identificador del thread actual (0, 1, 2, ...).
<code>int omp_get_num_threads()</code>	Devuelve el número total de threads activos en la región.

<code>int omp_get_max_threads()</code>	Devuelve el número máximo de threads posibles
<code>int omp_in_parallel()</code>	Retorna 1 si el código actual se ejecuta dentro de una región paralela, 0 en caso contrario.



Continuemos con otro video para explorar estas funciones adicionales de OpenMP.

Fundación Sadosky



Mirar en

Ejemplo 3:

```
#include
#include

int main() {
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        int total = omp_get_num_threads();
        int max = omp_get_max_threads();
        int paralelo= omp_in_parallel();
        printf("Soy el thread %d de %d Maximo=%d\n", tid, total,max);
        if(paralelo==1) printf("En region paralela\n");
    }
    if (omp_in_parallel())
        printf("Todavía en paralelo\n");
    else
        printf("Región paralela finalizada\n");
    return 0;
}
```



OpenMP permite crear regiones paralelas dentro de otras. Por defecto, el paralelismo anidado está deshabilitado por razones de eficiencia, pero puede activarse mediante:

```
omp_set_nested(1);
o
export OMP_NESTED=TRUE
```

Ejemplo 4:

```
#pragma omp parallel num_threads(2)
{
    printf("Nivel 1, thread %d\n", omp_get_thread_num());
    #pragma omp parallel num_threads(2)
    {
        printf(" Nivel 2, thread %d\n", omp_get_thread_num());
    }
}
```

En total, se ejecutarán 4 *threads* (2×2) si el *hardware* lo permite.

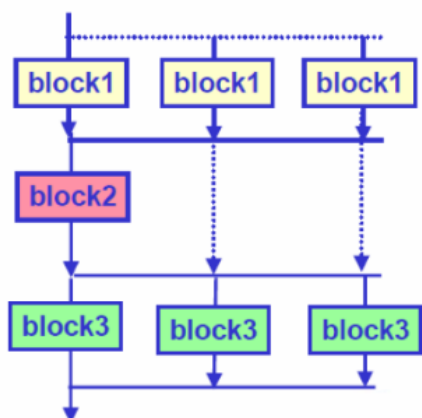


Buenas prácticas

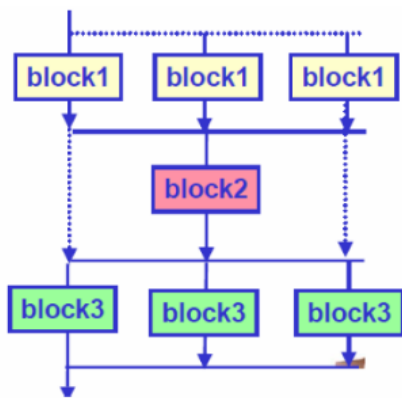
Es importante destacar que el paralelismo anidado puede aumentar la sobrecarga del sistema. Se recomienda solo en casos específicos, como algoritmos jerárquicos o tareas recursivas.

2.3 Directiva master y single

Dentro de una región paralela, a veces es necesario ejecutar una parte del código solo por un *thread* (típicamente el *master thread*). Para ello se usan las directivas:



```
#pragma omp master
Ejecutado únicamente por el thread 0 (sin barrera implícita).
```

`#pragma omp single`

Ejecutado por un *single thread* (con barrera al final, por defecto).

Veamos el ejemplo 5:

```
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    #pragma omp single
        printf("Inicialización hecha por un solo thread %d\n", tid);
    #pragma omp barrier
        printf("Todos los threads continúan después\n");
    #pragma omp master
        printf("Inicialización hecha solo por el thread %d\n", tid);
}
```

3. Sincronización y comunicación

Cuando varios *threads* trabajan sobre datos compartidos, es fundamental coordinar sus acciones para evitar resultados incorrectos. Esta coordinación se conoce como sincronización, y consiste en controlar cuándo y cómo los *threads* acceden a los recursos comunes (memoria, variables, archivos, etc.). OpenMP ofrece distintas herramientas de sincronización que permiten garantizar la coherencia de los datos y el orden de ejecución sin perder eficiencia.

3.1. Mecanismos de sincronización en OpenMP

OpenMP proporciona distintas directivas y funciones para sincronizar la ejecución y proteger los datos:

Mecanismo	Propósito principal
<i>Barrier</i>	Fuerza a todos los <i>threads</i> a esperar hasta que todos lleguen a ese punto.
<i>Critical</i>	Asegura que una sección de código sea ejecutada por un solo <i>thread</i> a la vez.
<i>Atomic</i>	Protege operaciones simples sobre variables compartidas (más eficiente que <i>critical</i>).
<i>Nowait</i>	Evita que los <i>threads</i> esperen al terminar una región paralela (región crítica, section, single, etc.)
<i>Lock / Unlock</i>	Sincronización manual y flexible mediante bloqueos explícitos.

Veamos cada uno de ellos en detalle.

Barrier o sincronización

Para comprender la sincronización, pensemos un ejemplo: en un primer paso, cada *thread* realiza cálculos sobre un subconjunto de columnas de una matriz y, en el siguiente, se usa la matriz completa en otra etapa del algoritmo. Sin la barrera, algunos *threads* podrían avanzar usando datos aún no calculados por otros. Entonces actúa *barrier*.



#pragma omp barrier hace que todos los *threads* esperen en ese punto hasta que todos lleguen. Ningún *thread* continúa antes de que todos los demás alcancen la barrera.

Veamos esta función en el ejemplo 7:

```
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    printf("Antes de la barrera - thread %d\n", tid);
    #pragma omp barrier
    printf("Después de la barrera - thread %d\n", tid);
}
```

Salida:

- Antes de la barrera - hilo 1
- Antes de la barrera - hilo 0
- Antes de la barrera - hilo 2
- Antes de la barrera - hilo 3
- Después de la barrera - hilo 2
- Después de la barrera - hilo 3
- Después de la barrera - hilo 1
- Después de la barrera - hilo 0

Sin la barrera, los mensajes podrían entremezclarse en cualquier orden. Con la barrera, todos los threads imprimen la primera parte antes de pasar a la segunda.

Critical o exclusión mutua

Para comprender critical pensemos el siguiente ejemplo: se desea insertar elementos en una estructura de datos compartida por todos los *threads* o escribir resultados en un archivo de salida. Esta directiva garantiza exclusión mutua, pero puede generar cuellos de botella si se usa en exceso.



La directiva *critical* define una sección crítica, es decir, un bloque que solo un *thread* puede ejecutar a la vez:

```
#pragma omp critical
{
    // código protegido
}
```

Veamos en el ejemplo 8 cómo se corrige el ejemplo 6:

```
#include <stdio.h>
#include <omp.h>

int main() {
    int x = 0;
    #pragma omp parallel for
    for (int i = 0; i < 1000; i++) {
        #pragma omp critical
        x++;
    }
    printf("Resultado correcto: %d\n", x);
    return 0;
}
```

Salida:

- Resultado correcto: 1000

Atomic o acumular

Se utiliza, por ejemplo, cuando se desea contar cuántos elementos cumplen una condición (por ejemplo cuantos elementos son mayores a 1) o acumular una suma global. Es más eficiente que *Critical*, pero solo se aplica a operaciones aritméticas o lógicas simples.



Se utiliza para operaciones simples (como sumas, restas, incrementos) porque es más eficiente que *Critical*.

`#pragma omp atomic`

Ejemplo 9:

```
#pragma omp parallel for
for (int i = 0; i < 1000; i++) {
    #pragma omp atomic
    x++;
}
```

Esta función tiene menor *overhead*. Su uso recomendado es para operaciones matemáticas simples sobre variables compartidas.

Nowait

Esta función se utiliza cuando se ejecuta un bucle (for) en paralelo seguido de tareas que no dependen del resultado global del bucle. Reduce el *overhead* de sincronización y mejora el rendimiento.



Por defecto, muchas directivas de OpenMP (como *for*, *sections* o *single*) incluyen una barrera implícita al final. Esto significa que todos los *threads* deben terminar esa región antes de continuar con la ejecución siguiente. La cláusula *nowait* elimina esa barrera, permitiendo que los *threads* que ya terminaron continúen sin esperar a los demás:

```
#pragma omp sections nowait
```

```
#pragma omp single nowait
```

```
#pragma omp for nowait
```

Locks o bloqueos

Los bloqueos se utilizan cuando se quiere proteger múltiples regiones de código no contiguas o implementar estructuras de datos avanzadas, como listas enlazadas o *buffers* compartidos).

OpenMP también permite manejar bloqueos explícitos mediante funciones de la biblioteca:

```
omp_lock_t lock;
```



```
omp_init_lock(&lock);
```

```
omp_set_lock(&lock);
```

```
omp_unset_lock(&lock);
```

```
omp_destroy_lock(&lock);
```

Ejemplo 10:

```
#include <stdio.h>
#include <omp.h>

int main() {
    int x = 0;
    omp_lock_t lock;
    omp_init_lock(&lock);

    #pragma omp parallel for
    for (int i = 0; i < 1000; i++) {
        omp_set_lock(&lock);
        x++;
        omp_unset_lock(&lock);
    }

    omp_destroy_lock(&lock);
    printf("Resultado = %d\n", x);
    return 0;
}
```

Esta directiva es más flexible que *Critical*, y es útil cuando se necesita proteger estructuras de datos específicas o secciones no contiguas del código. Sin embargo, requiere atención: olvidar liberar el *lock* puede provocar *deadlocks*.

4. Directivas Avanzadas

En este capítulo trabajaremos sobre directivas avanzadas dentro de OpenMP: la creación de regiones paralelas. Esto puede lograrse utilizando las siguientes cláusulas:

Cláusula	Descripción
private(var)	Cada <i>thread</i> tiene su copia independiente de var.
shared(var)	Variable compartida entre todos los <i>threads</i> .
reduction(op:var)	Combina valores parciales de todos los <i>threads</i> usando la operación op (+,-,*,min,max).
firstprivate(var)	Cada <i>thread</i> debe tener su propia copia privada de una variable, inicializada con el valor que tenía la variable original antes de entrar en la región paralela
lastprivate(var)	El valor final de la variable (el del último ciclo ejecutado) se copia de vuelta a la variable original después de salir de la región paralela.

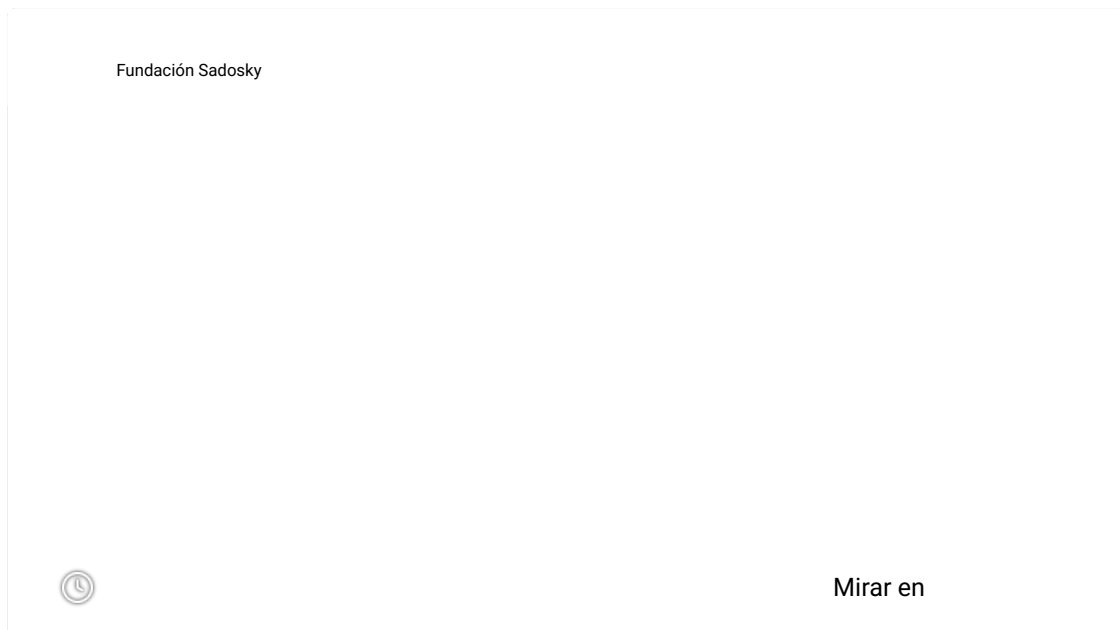
Al igual que en el capítulo anterior, veremos en detalle cada una de las cláusulas.

Shared o compartidas

Por defecto, las variables declaradas fuera de la región paralela son compartidas o *shared*, a menos que se indique lo contrario.



En el siguiente video se explica cómo utilizar variables compartidas y variables privadas:



Ejemplo 11:

```
t#include <stdio.h>
#include <omp.h>
int main() {
    int x = 10; // variable compartida por defecto

    #pragma omp parallel shared(x)
    {
        int tid = omp_get_thread_num();
        printf("Thread %d ve x = %d\n", tid, x);

        // todos los threads modifican la misma variable compartida
        x += tid;
    }

    printf("Valor final de x = %d\n", x);
    return 0;
}
```

Salida:

- Thread 0 ve x = 10
- Thread 1 ve x = 10
- Valor final de x = 11

La variable x está compartida por lo que todos los *threads* acceden a la misma dirección de memoria. Sin sincronización (*Critical*) habría una condición de carrera.

Private

Veamos el ejemplo N°12 con `private(var)`.

```
#include <stdio.h>
#include <omp.h>

int main() {
    int x = 10;

    #pragma omp parallel private(x)
    {
        int tid = omp_get_thread_num();
        x = tid; // cada thread tiene su propia versión de x
        printf("Thread %d: x = %d\n", tid, x);
    }

    printf("Valor final de x (fuera) = %d\n", x);
    return 0;
}
```

Salida:

- Thread 0: x = 0
- Thread 1: x = 1
- Valor final de x (fuera) = 10



Utilizando *private*, cada *thread* tiene su propia copia de la variable, inicializada sin valor (no se copia la original). Dentro de la región paralela, cada *thread* tiene su copia local de *x* no relacionada con la original. Los cambios dentro del bloque no afectan a la variable *x* externa.

Ahora veamos la diferencia entre *private* y *shared* en el mismo código con el ejemplo 13:

```
#include <stdio.h>
#include <omp.h>
int main()
{
    int shared_var = 100;
    int private_var = 200;

    #pragma omp parallel shared(shared_var) private(private_var)
    {
        int tid = omp_get_thread_num();
        private_var = tid * 10;
        #pragma omp critical
        {
            shared_var += tid;
            printf("Thread %d: shared_var = %d, private_var = %d\n",
                  tid, shared_var, private_var);
        }
    }
    printf("Valor final fuera de la región: shared_var = %d, private_var = %d\n",
          shared_var, private_var);

    return 0;
}
```

Salida:

- Thread 0: shared_var = 100, private_var = 0
- Thread 1: shared_var = 101, private_var = 10
- Valor final fuera de la región: = shared_var = 101, private_var = 200



Al usar la directiva *shared*, todos los *threads* acceden a la misma variable global. Los cambios son acumulativos y puede ocurrir condición de carrera. Al utilizar la directiva *private*, cada *thread* tiene su propia copia local y así no afecta al valor externo.

Veamos cómo se da esto en bucle con el ejemplo 14:

```
#include <stdio.h>
#include <omp.h>

int main() {
    int n = 5;
    int A[n];

    #pragma omp parallel for private(n)
    for (int i = 0; i < 5; i++)
    {
        n = i; // variable privada
        A[i] = n * 2;
        printf("Thread %d: A[%d] = %d (n local = %d)\n",
               omp_get_thread_num(), i, A[i], n);
    }

    printf("\n Después del bucle, n (fuera) no cambió: n = %d\n", n);
    return 0;
}
```

Salida:

- Thread 0: A[0] = 0 (n local = 0)
- Thread 0: A[1] = 2 (n local = 1)
- Thread 0: A[2] = 4 (n local = 2)
- Thread 1: A[3] = 6 (n local = 3)
- Thread 1: A[4] = 8 (n local = 4)

Después del bucle, *n* (fuera) no cambió *n* = 5. Cada *thread* tiene su propia copia de *n*, y el *n* del *main* no se modifica.

Reduction

En el ejemplo 17 vemos la directiva *reduction*:

```
#include <stdio.h>
#include <omp.h>
int main() {
    int n = 1000000;
    double suma = 0.0;

    #pragma omp parallel for reduction(+:suma)
    for (int i = 0; i < n; i++)
    {
        suma += 1.0;
    }

    printf("Resultado = %.0f\n", suma);
    return 0;
}
```

La salida es:

- Resultado = 1000000

La variable *suma* es privada para cada *thread*. Luego OpenMP combina los resultados. Así se evitan las condiciones de carrera sin necesidad de *critical*.

5. Distribución de trabajo (work-sharing)

Una vez creados los *threads* dentro de una región paralela, el siguiente paso es decidir cómo se reparte el trabajo entre ellos. OpenMP proporciona un conjunto de directivas de *work-sharing* que permiten dividir automáticamente las tareas o iteraciones de bucles entre los *threads* de un equipo, sin necesidad de control manual de índices o sincronización.

Estas estructuras determinan qué porción del código ejecuta cada *thread*, garantizando que cada parte del trabajo se ejecute una sola vez (a diferencia de la región paralela básica, donde todos los *threads* ejecutan lo mismo).

Concepto general

Una estructura de *work-sharing* no crea nuevos *threads*; simplemente distribuye el trabajo entre los *threads* ya activos dentro de una región paralela.

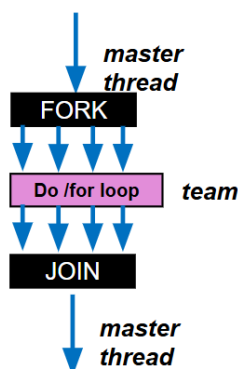
Las principales directivas son:

Directiva	Descripción
#pragma omp for	Divide las iteraciones de un bucle entre los <i>threads</i> .
#pragma omp sections	Asigna diferentes bloques de código a diferentes <i>threads</i> .
#pragma omp single	Hace que solo un <i>thread</i> ejecute un bloque.
#pragma omp master	Hace que solo el <i>master thread</i> ejecute el bloque.
#pragma omp task	Crea tareas independientes (paralelismo dinámico).

Directiva *For*

Es la más utilizada en OpenMP.

```
#pragma omp for
```



Esta directiva permite que las iteraciones de un bucle *for* se repartan automáticamente entre los *threads* disponibles. Representa el paralelismo de datos.

La imagen de la izquierda ilustra este funcionamiento siguiendo la estructura *fork-join* que vimos al inicio del módulo.



A continuación, en un nuevo video, vamos a ver cómo paralelizar un bucle *for*:

Fundación Sadosky



Mirar en

Veamos el ejemplo 18:

```
#include <stdio.h>
#include <omp.h>
int main()
{
    int n = 10;
    int A[n];

    #pragma omp parallel
    {
        #pragma omp for
        for (int i = 0; i < n; i++) {
            A[i] = i * 2;
            printf("Thread %d calculó A[%d] = %d\n", omp_get_thread_num(), i, A[i]);
        }
    }
    return 0;
}
```

Salida:

- Thread 1 calculó A[3] = 6
- Thread 1 calculó A[4] = 8
- Thread 1 calculó A[5] = 10
- Thread 0 calculó A[0] = 0
- Thread 0 calculó A[1] = 2
- Thread 0 calculó A[2] = 4
- Thread 2 calculó A[6] = 12
- Thread 2 calculó A[7] = 14
- Thread 3 calculó A[8] = 16
- Thread 3 calculó A[9] = 18

Cada *thread* procesa un subconjunto de iteraciones del bucle. El orden de ejecución no está garantizado (puede variar entre ejecuciones).

OpenMP permite especificar cómo se asignan las iteraciones a los *threads* mediante la cláusula `schedule()`:

Tipo de <i>Schedule</i>	Descripción	¿Cuándo usarlo?
<i>Static</i>	Asigna bloques fijos de iteraciones a cada <i>thread</i> (por defecto).	Cuando todas las iteraciones tienen similar costo.
<i>Dynamic</i>	Asigna bloques en tiempo de ejecución, a medida que los <i>threads</i> terminan.	Cuando las iteraciones tienen costo variable.

Veamos su aplicación con el ejemplo 19

```
#include <omp.h>
#include <stdio.h>
#include <unistd.h>
#define N 10
main ()
{
    int tid;
    int A[N];
    int i;
    for(i=0; i<N; i++) A[i]=-1;
    #pragma omp parallel for schedule(static,2) private(tid)
        for (i=0; i<N; i++){
            tid = omp_get_thread_num();
            A[i] = tid;
            usleep(1);
        }
    for (i=0; i<N/2; i++) printf (" %2d", i);    printf ("\n");
    for (i=0; i<N/2; i++) printf (" %2d", A[i]);printf ("\n\n\n");
    for (i=N/2; i<N; i++) printf (" %2d", i);    printf ("\n");
    for (i=N/2; i<N; i++) printf (" %2d", A[i]);printf ("\n\n\n");
}
```

Cada *thread* trabaja con *batch* de dos elementos por vez, respetando el orden de acceso al arreglo de acuerdo con el identificador del *thread* (*load balancing* estático).

Salida:

```
0 1 2 3 4 5 6 7 8 9
0 0 1 1 0 0 1 1 0 0

10 11 12 13 14 15 16 17 18 19
1 1 0 0 1 1 0 0 1 1
```

Al cambiar el tipo de *scheduling* a dinámico:

```
#pragma omp parallel for schedule(dynamic,2) private(tid)
```

cada *thread* trabaja con batch de dos elementos por vez, y el primero que se libera toma el siguiente *batch* (*load balancing* dinámico).

Salida:

```
0 1 2 3 4 5 6 7 8 9
0 0 1 1 1 1 0 0 0 0
```

```
10 11 12 13 14 15 16 17 18 19
1 1 1 1 0 0 1 1 0 0
```

Cuando se paraleliza un bucle, las iteraciones pueden ejecutarse en cualquier orden. Si se necesita preservar el orden secuencial (por ejemplo, para imprimir resultados), se usa: `#pragma omp ordered`

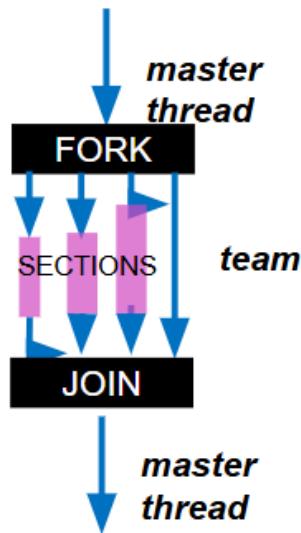
Algo importante es que no se puede paralelizar un bucle cuyo cuerpo contiene *break*, *return* o *exit* en un *parallel for*. Por ejemplo:

```
for (i=0;i<n;i++) {
    a[i] = 2.3*i;
    if (a[i] < b[i]) break;
}
```

Sections: paralelismo de tareas independientes

Ahora veremos la función que permite dividir el código en bloques distintos, ejecutados por diferentes *threads*: `#pragma omp sections`. Es útil cuando se tienen tareas heterogéneas, no iteraciones.

Sections divide el trabajo en secciones discretas, y cada una se ejecuta por un *thread*. Se trata de paralelismo de tareas, como se aprecia en el siguiente diagrama:



Veamos entonces el código en el ejemplo 21:

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel sections
    {
        #pragma omp section
        { printf("Tarea 1 ejecutada por thread %d\n", omp_get_thread_num()); }

        #pragma omp section
        { printf("Tarea 2 ejecutada por thread %d\n", omp_get_thread_num()); }

        #pragma omp section
        { printf("Tarea 3 ejecutada por thread %d\n", omp_get_thread_num()); }
    }
    return 0;
}
```

Cada section es ejecutada una sola vez, por un *thread* distinto del equipo.

Cierre



¡Felicitaciones! Llegaste hasta el final del módulo 2.

En este módulo, abordamos de manera práctica cómo aprovechar el paralelismo en arquitecturas de memoria compartida mediante el uso de OpenMP. Introducimos el modelo de *threads*, la creación de regiones paralelas y el uso de directivas básicas para distribuir trabajo de manera eficiente.

Además, revisamos los principales mecanismos de sincronización, así como las buenas prácticas para evitar

condiciones de carrera y sobrecostos innecesarios. Con estos fundamentos, podemos comenzar a implementar programas *multithread* y analizar su comportamiento, estableciendo la base para abordar técnicas más avanzadas de programación paralela en sistemas *multicore* y entornos HPC.



¡Y ya es hora de practicar lo aprendido!

Dirigite a la página de ejercitación haciendo [clic aquí](#). No te olvides de que es necesario ver esta actividad para poder acceder al siguiente módulo.